

APM Incident Triage Assistant

Technical proposal | Take-home assessment | 6 October 2026

1. Problem and boundaries

An operator spends time moving between payment metrics, provider status pages, tickets and logs before deciding whether an alert represents a real incident. I would automate evidence collection and use AI to turn verified evidence into a short, reviewable incident brief. The first release would support decisions without changing payment behaviour.

Assumptions: read-only access to aggregated metrics, approved log extracts and a provider ownership directory; a sandbox ticket system; synthetic data for this demonstration. Thresholds below are illustrative and require calibration by payment method, country and traffic profile. No measured operational improvement is claimed.

Use rules for counters, freshness, thresholds, deduplication and routing. Use AI for summarising heterogeneous evidence, suggesting explicitly uncertain hypotheses, and retrieving relevant approved runbook steps. A human confirms severity, escalation, customer communications, recovery and every production change.

2. Three practical AI use cases

A. Incident evidence brief

Input: validated metrics, response-code groups, normalized provider status and sanitized support signals. AI action: produce an evidence-linked summary, possible explanations and missing checks. Output: draft incident brief. User: APM on-call operator. Value: less manual copying and faster initial assessment; measure time to an approved brief.

B. Runbook assistance

Input: reviewed incident scope plus versioned, access-controlled runbooks. AI action: retrieve relevant sections and propose read-only diagnostic checks, citing document IDs and versions. Output: checklist for the operator. User: Tech Ops investigator. Value: more consistent investigation and fewer missed steps; measure accepted suggestions and unsafe/unsupported suggestions.

C. Handover and resolution summary

Input: ticket timeline, verified actions and recovery metrics. AI action: draft shift handover or incident review, distinguishing confirmed facts from open questions. Output: reviewed handover and post-incident draft. User: incoming on-call engineer and incident owner. Value: less context loss; measure preparation time and factual corrections.

Why start with use case A?

It addresses a frequent operational bottleneck, needs no payment write access, and can be compared with the current manual process. Runbook assistance and handovers can follow once evidence quality and operator trust are established.

Detailed workflow and decisions

3. Trigger and data contract

Run every five minutes, or when a monitoring alert or support escalation arrives. Normalize provider, method, country and currency; record source timestamps and evidence IDs. Deduplicate payment attempts upstream. Calculate outcomes only for attempts old enough to have completed under the method-specific SLA. Keep recent pending attempts separate. An instant transfer and a voucher payment must not share the same maturity threshold.

The demo uses a 15-minute maturity SLA and a matched baseline success rate supplied by the metrics adapter. A production baseline should use comparable time-of-day/day-of-week cohorts, exclude known outages, and account for changes in merchant mix. Missing or unreliable baselines require manual review.

4. Decision sequence

- Validate data: counters must be nonnegative and reconcile; metrics cannot be future-dated or more than three minutes old. Invalid or stale metrics generate an observability review, not a provider-outage claim.
- Check degradation: at least 100 matured attempts, baseline at least 80%, success-rate drop of at least 10 percentage points, and two consecutive bad windows. Low-volume signals remain visible for manual review; the threshold does not prove they are harmless.
- Propose severity: P1 for confirmed threshold breaches affecting at least five merchants and at least 200 failed or overdue attempts; otherwise P2. These are demonstration thresholds, not company policy. Suspected financial-integrity or security issues need a separate urgent human escalation path.
- Correlate supporting signals: a provider status warning increases evidence of a provider issue but cannot prove root cause. An operational status page cannot clear a provider. Check outbound requests, network errors, callbacks and platform changes.
- Prepare draft: group signals by provider/method/country/currency and update an existing active incident. Route initially to APM Tech Ops using the ownership directory. AI proposes a short summary and next checks; it cannot decide ownership or change severity.
- Human decision: the operator reviews evidence and approves, edits or rejects escalation. Customer/provider messages and payment configuration changes always require a separate authorized approval.

5. Evidence strength and root cause

Report evidence strength as a qualitative assessment, not a made-up probability. A strong signal needs fresh, consistent metrics plus independent corroboration. Metrics alone establish impact, not responsibility. Confirm provider-side versus platform-side causes only after tracing the request/response boundary and reconciling affected outcomes. The prototype deliberately leaves root cause unconfirmed.

Automation, controls and operations

6. End-to-end production design

Sources → read-only adapters → normalized evidence store → deterministic validation and incident rules → constrained model draft → schema/reference checks → operator review → approved ticket/escalation → recovery monitoring → human closure.

Monitoring connectors supply rates and maturity cohorts. A provider adapter normalizes status pages, while support connectors redact and extract relevant facts. The ownership directory supplies the responsible team. Persistent incident state supports deduplication, owner acknowledgement and reminder timers. Connector retries use backoff; failed sources are marked unavailable rather than silently treated as healthy.

After an approved escalation, route acknowledgement failures to the incident lead according to an agreed timer. Require three healthy monitoring windows plus checks on pending backlog and financial outcomes before proposing closure. A human closes the incident. The demo stops at local drafts; connectors, persistence across runs, approval execution and recovery logic are future implementation steps.

7. Guardrails and auditability

- Least privilege: no credentials allowing refunds, retries, routing changes or production writes. The model has no execution tools.
- Privacy: send only approved aggregates and redacted evidence. Exclude PAN, CVV, secrets and unnecessary personal data. Apply the organization's approved model, retention and data-residency policy before using real data.
- Untrusted content: provider pages, logs and ticket bodies are data, never instructions. Keep them separate from system instructions; allowlist source domains and output fields. Human review remains necessary even after validation.
- Output validation: require structured JSON and valid evidence references. Reference checks do not prove that a statement is true. Reject invalid drafts and fall back to a deterministic evidence report; do not delay paging for an AI response.
- Audit: retain source timestamps, rule and prompt versions, model version where applicable, input hashes, draft revisions, reviewer identity, approval decision and action outcome. Protect logs with access controls and a retention policy.

8. Cost and latency

Use deterministic rules to filter routine signals and call the model once per meaningful incident update. Cache briefs by evidence hash. Start with a small model for summaries; consider a larger model only when measured review quality justifies its cost. Target a ten-second draft timeout and one bounded retry, then return the rules-only brief. These are proposed targets, not benchmark results.

Prototype, validation and rollout

9. What the prototype actually does

demo.py is executable Python using the standard library. It validates synthetic aggregate counters, excludes recent pending attempts from the mature cohort, detects sustained degradation, proposes severity and ownership, deduplicates within one run, and writes local JSON drafts with evidence and an audit record. No API keys, payment data or external access are needed.

AI MODE: OFFLINE MOCK. The AI function is a deterministic test double, not a live language model. It demonstrates the interface, prompt boundary and structured draft validation. It does not demonstrate model reasoning quality or real integrations. The supplied PROMPT and model_payload define the boundary for a future approved model adapter.

Run

```
python demo.py --self-test
python demo.py --output demo_output.json
```

Expected scenarios

- Provider degradation: 1,000 matured attempts, 650 successful, baseline 98%, two bad windows. A 33-percentage-point drop produces a P1 draft because the illustrative impact threshold is also met. Provider responsibility remains a hypothesis.
- Normal asynchronous pending: 980 successes out of 1,000 matured attempts; 500 additional recent pending attempts are excluded. No incident ticket is proposed.
- Stale metrics: a twenty-minute-old snapshot requests an observability review; the incident severity is unassessed.
- Duplicate signal: a second matching degradation updates the existing draft rather than creating another one.

The included self-test also checks invalid counters, low volume, rejection of unknown evidence references, and the absence of external side effects. The simulated clock is fixed for reproducibility. Deduplication is in-memory and expires when the process exits.

10. Rollout and success measures

Start in shadow mode on historical, sanitized incidents and a live read-only sample. Establish a manual baseline before setting improvement targets. Compare time-to-triage and time-to-correct-owner (median and p90); measure false incident candidates and missed incidents against human-reviewed labels. Track factual correction rate, operator acceptance, unauthorized actions (target zero), cost per reviewed brief, and end-to-end draft latency.

MTTR is a downstream outcome influenced by incident mix and provider recovery; compare similar incident classes rather than attributing every improvement to AI. Capture reviewer corrections with reason codes, review them weekly, and version prompt/rule changes. Do not let feedback automatically update production rules. Expand to approved ticket creation only after a reviewed pilot demonstrates safe and useful results.